

Query Optimization: Materialized Views

1 Introduction

A **Materialized View** is a view whose contents are computed and stored physically in the database. Unlike regular views, which are virtual and computed on-the-fly, materialized views provide a persistent snapshot of the data.

View vs. Materialized View

```
create materialized view dept_total_salary as
select dept_name, sum(salary) from instructor
group by dept_name;
```

Accessing this materialized view avoids scanning all instructor tuples and summing salaries repeatedly.

2 Materialized View Maintenance

Keeping a materialized view up-to-date with underlying data is known as **maintenance**.

2.1 Maintenance Methods

- **Manual Code:** Explicitly updating the view in application logic (error-prone).
- **Triggers:** Automatic execution during DML operations on base relations.
- **Immediate Maintenance:** Updated within the same transaction as the base data change.
- **Deferred Maintenance:** Updates are collected and applied later (periodic/nightly).

3 Incremental View Maintenance

Changes to relations (inserts and deletes) are called the **differential**. We denote inserts as i_r and deletes as d_r . Updates are treated as a *delete(old)* followed by an *insert(new)*.

3.1 Join Operation ($v = r \bowtie s$)

Consider an insert i_r into r :

$$v_{new} = r_{new} \bowtie s = (r_{old} \cup i_r) \bowtie s = (r_{old} \bowtie s) \cup (i_r \bowtie s)$$

$$v_{new} = v_{old} \cup (i_r \bowtie s)$$

Meaning: We only need to join the **new tuples** with the other relation and add them to the view.

3.2 Selection and Projection

- **Selection** ($\sigma_\theta(r)$): $v_{new} = v_{old} \pm \sigma_\theta(i_r/d_r)$.
- **Projection** ($\Pi_A(r)$): Requires maintaining a **derivation count**.

Projection Maintenance

For each tuple $t \in \Pi_A(r)$, store a count of base tuples that project to t .

- **Insert**: If t is present, increment count; else add t with count 1.
- **Delete**: Decrement count. Remove t only if count reaches 0.

4 Aggregation Maintenance

- **Count**: Similar to projection count logic.
- **Sum**: Maintain group sum and a **tuple count**.

Why track count for Sum?

Without a count, we cannot distinguish between a group whose sum happens to be 0 and a group that should be deleted because all its tuples were removed.

- **Average**: Maintain sum and count, then compute $avg = \text{sum} / \text{count}$.
- **Min / Max**: On delete, if the min/max tuple is removed, a **scan** of the group is required to find the new extreme value. An ordered index on (G, B) can optimize this.

5 Handling Expressions

Incremental changes can propagate bottom-up through an expression tree. For $E = E_1 \bowtie E_2$: If ΔE_1 is the change to E_1 , the change to the join result is $\Delta E_1 \bowtie E_2$.

6 Optimizer and Materialized Views

6.1 Query Rewriting

The optimizer recognizes when a query (e.g., $r \bowtie s \bowtie t$) can be answered more efficiently using an existing view ($v = r \bowtie s$): $v \bowtie t$.

6.2 Replacing Views with Definitions

If a view v lacks an index but base relation r has one on selected attribute A , $\sigma_{A=10}(v)$ might be faster as $\sigma_{A=10}(r) \bowtie s$.

7 Materialized View Selection

”What is the best set of views to materialize?” This is part of **database tuning**.

Tuning Tools

- **Microsoft SQL Server:** Database Tuning Assistant.
- **IBM DB2:** Design Advisor.
- **Oracle:** SQL Tuning Wizard.